

第一模块：C语言与内存模型（内核级超详细版 · 嵌入式装备方向）

本模块只聚焦：C语言在嵌入式系统中的底层语义、内存模型与编译器行为。

不包含：系统启动流程、中断机制、RTOS等内容。

目标不是会写语法，而是能够解释： - 编译器到底生成了什么 - 指针和内存如何真实工作 - 为什么某些代码在优化后会“失效”

一、数据类型与位宽（底层基础中的基础）

1.1 为什么不能随使用 int

C标准没有规定 int 一定是 32 位。

不同平台可能：

- 16 位
- 32 位
- 64 位

嵌入式工程原则：

必须使用 `stdint.h`

```
uint8_t
uint16_t
uint32_t
```

原因：

- 与寄存器位宽一致
- 与通信协议一致
- 与DMA搬运长度一致

1.2 printf 的真实风险（高频底层问题）

错误示例：

```
uint32_t a;  
printf("%d",a);
```

原因：

printf是可变参数函数，不检查类型。

解决：

```
#include <inttypes.h>  
printf("%u" PRIu32,a);
```

二、指针与内存访问（嵌入式核心能力）

2.1 指针本质

指针不是“变量”，而是：

👉 一个地址。

CPU只认识地址，不认识变量名。

2.2 指针算术（很多人理解错）

```
uint32_t* p;  
p+1
```

实际增加：

```
+4 字节
```

因为按类型大小移动。

2.3 强制类型转换的真实风险（strict aliasing）

错误写法：

```
float f = *(float*)&u32;
```

在 -O 2 优化下可能被编译器重排。

正确方式：

```
memcpy(&f,&u32,4);
```

原因：

编译器默认不同类型指针不会指向同一内存。

三、内存布局（RAM中的真实世界）

3.1 三大区域

- stack（栈）
- heap（堆）
- static（静态区）

栈特点

- 自动分配
- 容量有限
- 溢出无提示

堆特点

- 动态申请
- 可能产生碎片

静态区

- 生命周期贯穿整个程序
-

3.2 为什么大数组不要放栈上

```
uint8_t buf[4096];
```

函数退出栈空间释放。

如果栈只有 2 KB会直接破坏内存。

四、结构体对齐与padding（极高频底层点）

4.1 为什么 sizeof 会变大

```
struct A{  
    char a;  
    int b;  
};
```

结果通常是 8 字节而不是 5 字节。

原因：

CPU要求对齐访问。

4.2 packed 的风险

```
__attribute__((packed))
```

虽然减少空间，但可能导致：

- 非对齐访问
 - 性能下降
 - 异常
-

五、volatile 的真正含义（99%人理解不完整）

5.1 volatile 做了什么

禁止编译器：

- 删除访问
 - 缓存寄存器
-

5.2 volatile 没有做什么

不能保证：

- 原子性

- 多任务安全
 - 执行顺序
-

5.3 正确使用场景

- 外设寄存器
 - ISR共享变量
 - DMA缓冲区
-

六、const 与指针组合（底层必问）

```
const int *p
int * const p
```

区别：

- 前者内容不可改
 - 后者地址不可改
-

七、函数调用与栈帧模型（语言层底层机制）

函数调用时发生：

- 参数压栈或进寄存器
- LR保存返回地址
- 分配局部变量

嵌入式问题来源：

- 递归
 - 局部变量过大
-

八、未定义行为（UB）——偶现Bug真正来源

常见UB：

- 野指针
- 数组越界
- printf类型错误
- 有符号溢出

UB特点：

- 编译通过
 - 偶尔出错
-

九、并发下的共享变量（语言层风险）

volatile 只能解决：

👉 编译器优化问题

不能解决：

👉 中断竞争

需要：

- 临界区
 - 原子操作
-

十、高频底层考点（标准答案版）

Q 1：volatile 的作用是什么？

标准答案：

防止编译器优化访问，保证每次真实读写内存，但不保证线程安全。

Q 2：为什么嵌入式不建议频繁 malloc？

标准答案：

产生碎片且耗时不可预测，影响实时性。

Q 3：struct 为什么会变大？

标准答案：

因为内存对齐产生padding。

Q 4 : strict aliasing 是什么?

标准答案:

编译器假设不同类型指针不别名，错误强转可能触发优化错误。

十一、工程注意事项（真正经验）

- 1) 不要猜类型位宽。
 - 2) 不要随意指针强转。
 - 3) volatile ≠ 线程安全。
 - 4) 栈大小必须评估。
 - 5) printf慎用。
-

十二、深度练习（建议一定做）

- 打开 -O 2 观察 volatile 汇编差异
 - 写一个错误 aliasing 示例再修复
 - 计算多个 struct 的 sizeof
-

本模块完成后，你应该具备：

能从“编译器 + 内存模型”的角度理解C代码，而不是只会写语法。